PHASE 3: SUBMISSION CHECKLIST/SIGNOFF SHEET

GROUP#: 1 **GROUP NAME:** Dashers

Deliverables:

- [Revised] Description of the organization
- [Revised] ER diagram with min/max specifications
- [Revised] Constraints not in ER diagram
- [Revised] Relational Schema with Referential Integrity
- [Revised] Queries with brief description
- [Revised] DML + DDL + SQL statements
- [Revised] Completed PostgreSQL Implementation (Attached Team SQL script)
- Completed ORM Implementation

Assessment:

- Group Status Report

We have each reviewed the contents of this deliverable.

Phase Leader	Abinaya Ramesh	_____
Phase Recorder	Zubair Zaki	_____
Phase Checker	Lingjie Liao	_____
Technical Advisor	Kellie Clinton	_____

1. Introduction

This project, the Supermarket Management System, is designed to provide a centralized and structured repository for core business data, including customer transactions, product inventory, and employee information. The primary focus of this project is to establish a foundation for a comprehensive system that can support the complex data requirements of a modern retail environment.

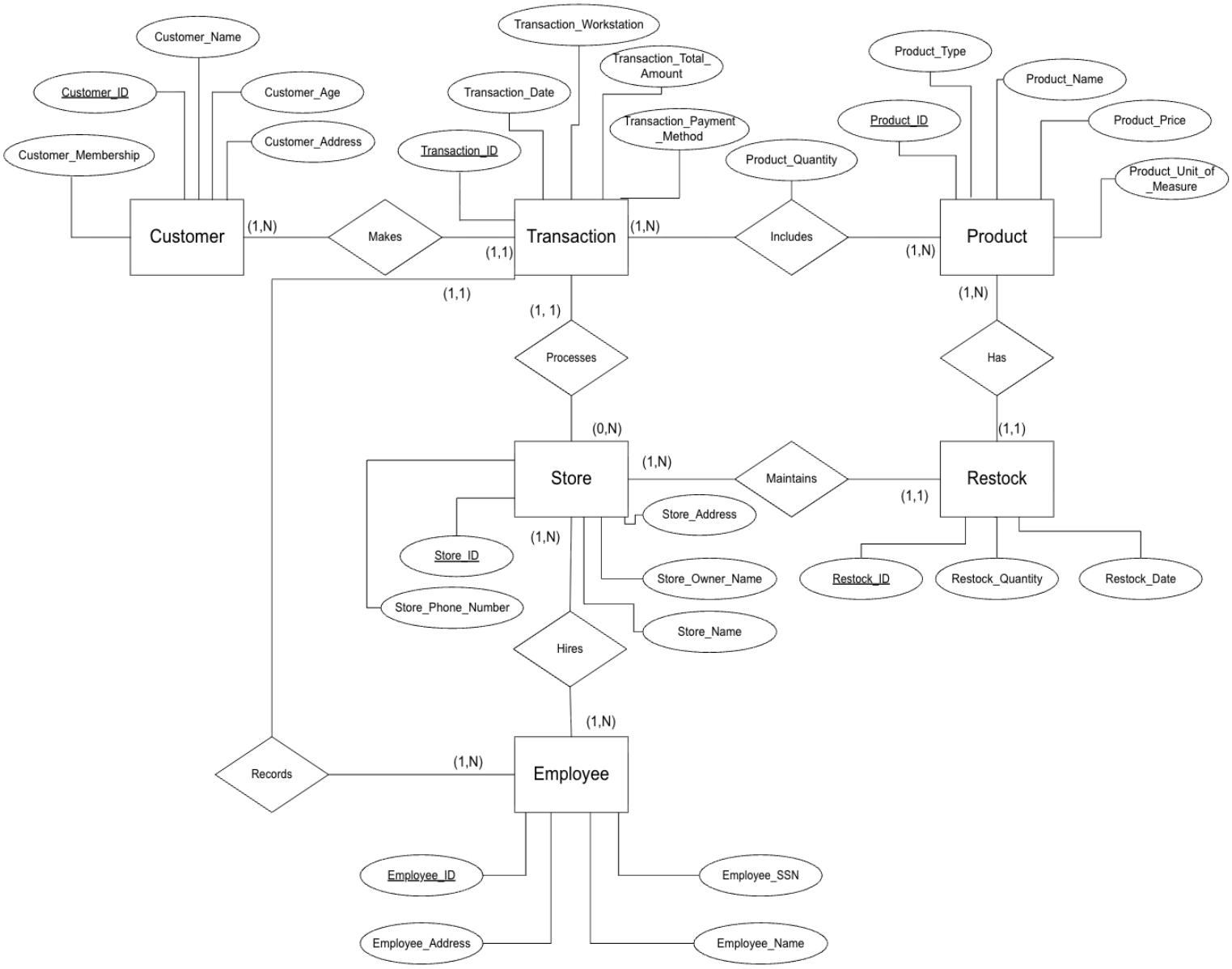
2. Description of the Supermarket Database

The Supermarket Database (SUPERMARKET) stores essential information required to manage the day-to-day operations of a retail store. The database contains data about customers, products, transactions, employees, stores, and inventories, ensuring that all activities within the supermarket are properly recorded and interconnected.

- Customers: Customers are represented in the database with a unique Customer_ID, along with attributes such as Customer_Name, Customer_Age, Customer_Address, and Customer_Membership. Each customer can make one to many transactions, and one transaction can only be made by one to one customer.
- Products: Products are uniquely identified by a Product_ID and described by attributes such as Product_Name, Product_Type, Product_Price, and Product_Unit_of_Measure. Products can be included by showing the Product_Quantity by one to many transactions and a transaction can include one to many products.
- Transactions: Transactions record purchase activities carried out in the supermarket. Each transaction has a unique Transaction_ID and other details such as Transaction_Workstation, Transaction_Date, Transaction_Total_Amount, Transaction_Payment_Method. One transaction can only be made by one to one customer, but one customer can make one to many transactions. One Employee can record one to many transactions, but one transaction can only be recorded by one to one employee. Each transaction can include one to many products, and each product includes one to many transactions.
- Stores: Each store in the organization is uniquely identified by a Store_ID and includes attributes such as Store_Address, Store_Phone_Number, Store_Name and Store_Owner_Name. Stores will hire one to many employees, one employee can be hired one to many stores. Each store maintains one to many restocks, but one restock is maintained one to one store. Stores can process zero to many transactions, but a transaction can only process one to one store.
- Employees: Employees are tracked in the database with a unique primary Employee_ID and unique candidate Employee_SSN. Another attribute Employee_Address and Employee_Name is included. Employees are hired by one to many stores and one store can hire one to many employees. An employee records one to many transactions, while a transaction is recorded by one to one employee.
- Restock: Restocks are uniquely identified by a Restocks_ID and contain attributes such as Restocks_Quantity and Restock_Date. Each restock is maintained by one to one store, but each store can maintain one to many restocks. Each restock has one to one product, and each product has one to many restocks.

3. ER Diagram

The figure below shows the ER diagram of the Supermarket Database (SUPERMARKET).



4. ER Diagram Uncaptured Constraints

The following is a list of constraints that are not captured by the ER diagram of SUPERMARKET:

- The product price and employee salary must be positive and in dollars.
- Employee SSN must be unique.
- There is no self checkout, thus each transaction has a physical employee (not virtual).
- Employee can work at one store at a time

5. Relational Schema

This section provides the relational schema with referential integrity and the relational table details.

5.1 Relational Schema with Referential Integrity

Store (Store_ID, Store_Address, Store_Phone_Number, Store_Name, Store_Owner_Name)

Employee (Employee_ID, Employee_Address, Employee_Name, Employee_SSN)

Hires (Store_ID, Employee_ID)
Foreign key (Store_ID) references Store (Store_ID)
Foreign key (Employee_ID) references Employee (Employee_ID)

Customer (Customer_ID, Customer_Name, Customer_Membership, Customer_Address, Customer_Age)

Product (Product_ID, Product_Type, Product_Name, Product_Price, Product_Unit_of_Measure)

Transaction (Transaction_ID, Transaction_Workstation, Transaction_Date, Transaction_Total_Amount, Transaction_Payment_Method, Employee_ID, Customer_ID, Store_ID)
Foreign key (Employee_ID) references Employee (Employee_ID)
Foreign key (Customer_ID) references Customer (Customer_ID)
Foreign key (Store_ID) references Store (Store_ID)

Includes (Transaction_ID, Product_ID, Product_Quantity)
Foreign key (Transaction_ID) references Transaction (Transaction_ID)
Foreign key (Product_ID) references Product (Product_ID)

Restock (Restock_ID, Restock_Date, Restock_Quantity, Store_ID, Product_ID)
Foreign key (Store_ID) references Store (Store_ID)
Foreign key (Product_ID) references Product (Product_ID)

5.2 Relational Table Details

The relational schema given in Section 5.1 was mapped into the following tables in the Supermarket Database (SUPERMARKET). Primary keys have been underlined. Tables that have multiple attributes underlined represent composite keys.

Table Name	Attribute	Description
Store	Store_ID	Unique Store ID
	Store_Address	Address of the Store
	Store_Phone_Number	Phone Number of Store
	Store_Name	Name of Store
	Store_Owner_Name	Name of the Store Owner
Employee	Employee_ID	Unique Employee ID
	Employee_Address	Address of Employee
	Employee_Name	Name of Employee
	Employee_SSN	Social Security Number of Employee
Hires	Store_ID	Unique Store ID which Hires Employee
	Employee_ID	Unique Employee ID that is Hired by Store
Customer	Customer_ID	Unique Customer ID
	Customer_Name	Name of the Customer
	Customer_Membership	Membership of Customer
	Customer_Address	Address of Customer
	Customer_Age	Age of Customer

Table Name	Attribute	Description
Product	Product_ID	Unique Product ID
	Product_Type	Type of Product
	Product_Name	Name of Product
	Product_Price	Price of Product
	Product_Unit_of_Measure	Unit of Measure of Product
Transaction	Transaction_ID	Unique Transaction ID
	Transaction_Workstation	Workstation of Transaction
	Transaction_Date	Date of Transaction
	Transaction_Total_Amount	Total Amount of Transaction
	Transaction_Payment_Method	Payment Method of Transaction
	Employee_ID	ID of Employee Recording the Transaction
	Customer_ID	ID of Customer Making Transaction
	Store_ID	ID of Store Processing Transaction
Includes	Transaction_ID	Unique ID of Transaction Including that Product
	Product_ID	Unique ID of the Product which Includes Transaction
	Product_Quantity	Quantity of Product
Restock	Restock_ID	Unique Restock ID
	Restock_Date	Date of Restock
	Restock_Quantity	Quantity of Restock
	Store_ID	ID of Store Maintaining the Restock
	Product_ID	ID of Product that has the Restock

6. Queries

Query Name	Description	Output	Relations Accessed	Name
Sum and Count of Transactions for each Employee per Store	Calculate the biggest transaction each employee per store will ring up that is more than $5 on 01/01/2025. Count how many transactions each employee processed.	Store_Name Employee_Name Total_Transaction_Amount Employee_Wrung_Up_Count	Transaction (Records) Employee (Processes) Store	Kellie Clinton
Transaction Summary with Total Value	Calculates, for each transaction, the number of distinct products, total quantity of items, and total value based on product prices. The query joins Includes, Transaction, and Product tables, groups results by transaction, and displays the top 10 transactions ordered by total value.	Transaction_ID Transaction_Date Num_Products Total_Items Total_Value Product_List	Includes Transaction Product	Lingjie Liao
Product Restock Summary with Total, Value and Average Batch Quantity	It calculates the total quantity restocked, the total value of the restocks (by multiplying product price with restocked quantity), the average quantity per restock batch and the number of restock events for each product. The data is grouped by product and displayed in descending order of total value, and then in ascending order of product name for clarity.	Product_ID Product_Name Product_Price total_Quantity total_value avg_batch_qty restock_events	Restock(Entity) (Has) Product(Entity)	Zubair Zaki
Customer Spending Summary	Print the total amount spent and average spending per transaction, the total number of distinct products bought, and the total number of transactions for each customer, which is sorted by total spent in descending order.	Customer_Name, Customer_Membership, Total_Spent, Avg_Spent_Per_Transaction, Distinct_Products_Bought, Total_Transactions	Customer (Records), Transaction (Records), Includes (Processes), Product (Records)	Abinaya Ramesh

7. DDL + DML + SQL (Including output screenshots)

The following is an SQL definition of the tables for the SUPERMARKET Database.

```
CREATE TABLE Transaction (
Transaction_ID integer NOT NULL,
Transaction_Date date,
Transaction_Workstation varchar(20),
Transaction_Total_Amount numeric,
Transaction_Payment_Method varchar(20),
Employee_ID integer,
Store_ID integer,
Customer_ID integer,
PRIMARY KEY(Transaction_ID),
FOREIGN KEY (Employee_ID) REFERENCES Employee(Employee_ID),
FOREIGN KEY (Store_ID) REFERENCES Store(Store_ID),
FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID));

CREATE TABLE Store(
Store_ID integer,
Store_Phone_Number varchar(20),
Store_Address varchar(255),
Store_Name varchar(20),
Store_Owner_Name varchar(50),
PRIMARY KEY (Store_ID));

CREATE TABLE Employee(
Employee_ID integer,
Employee_SSN varchar(20),
Employee_Address varchar(255),
Employee_Name varchar(20),
PRIMARY KEY (Employee_ID));

CREATE TABLE Product(
Product_ID integer,
Product_Type varchar(50) NOT NULL,
Product_Name varchar(40) NOT NULL,
Product_Price numeric,
Product_Unit_Of_Measure varchar(40),
PRIMARY KEY (Product_ID));

CREATE TABLE Includes(
Transaction_ID integer,
Product_ID integer,
Product_Quantity integer,
PRIMARY KEY (Transaction_ID, Product_ID),
FOREIGN KEY (Transaction_ID) REFERENCES Transaction (Transaction_ID),
FOREIGN KEY (Product_ID) REFERENCES Product (Product_ID));

CREATE TABLE Restock(
Restock_ID integer,
Store_ID integer,
Product_ID integer,
Restock_Quantity integer,
Restock_Date date,
PRIMARY KEY (Restock_ID),
FOREIGN KEY (Store_ID) REFERENCES Store (Store_ID),
FOREIGN KEY (Product_ID) REFERENCES Product (Product_ID));

CREATE TABLE Customer (
Customer_ID integer PRIMARY KEY,
Customer_Name VARCHAR(50) NOT NULL,
Customer_Membership VARCHAR(20) CHECK (Customer_Membership IN
('Gold', 'Silver', 'Bronze')),
Customer_Address varchar(100),
Customer_Age integer CHECK (Customer_Age > 0));

INSERT INTO Employee VALUES
(0001, '999-88-7777', '202 Broadway St', 'Julie Pumpkin'),
(0002, '212-63-4512', '144 Colfax Rd', 'Nancy Lemon'),
(0003, '747-00-8539', '555 Luna Rd', 'Tom Schwarts'),
(0004, '963-52-7219', '998 Plum Grove St', 'Rachel Lime'),
(0005, '417-77-8860', '231 Heather Ct', 'Dan Arnold'),
(0006, '551-27-9437', '741 Kennedy Rd', 'Sally Seashell');


INSERT INTO Customer VALUES
(001, 'Alice', 'Gold', '123 Main Street, Chicago', 28),
(002, 'Bob', 'Silver', '45 Lake View Drive, Chicago', 34),
(003, 'Carol', 'Gold', '789 Oak Lane, Evanston', 26),
(004, 'David', 'Bronze', '56 Pine Road, Naperville', 40),
(005, 'Emma', 'Silver', '91 River Street, Aurora', 31),
(006, 'Frank', 'Gold', '32 Maple Avenue, Skokie', 29),
(007, 'Grace', 'Bronze', '210 Hilltop Street, Chicago', 37);


INSERT INTO Transaction VALUES
(0001, ('01/01/2025'), 'WS-001', 60, 'Credit Card', 0001, 0001,001),
(0002, ('01/01/2025'), 'WS-002', 10, 'Check', 0002, 0004,002),
(0003, ('01/01/2025'), 'WS-003', 45, 'Credit Card', 0003, 0004,003),
(0004, ('01/01/2025'), 'WS-004', 105, 'Credit Card', 0002, 0004,001),
(0005, ('01/01/2025'), 'WS-005', 25, 'Direct Deposit', 0005, 0006,004),
(0006, ('01/01/2025'), 'WS-005', 3, 'Credit Card', 0005, 0006,005),
(0007, ('01/01/2025'), 'WS-003', 112, 'Check', 0004, 0004,002),
(0008, ('01/01/2025'), 'WS-005', 56, 'Cash', 0005, 0006,003),
(0009, ('01/01/2025'), 'WS-001', 47, 'Credit Card', 0005, 0006,007),
(0010, ('01/02/2025'), 'WS-002', 87, 'Cash', 0005, 0006, 006);

INSERT INTO Store VALUES
(0001, '1+555-123-3456', '123 Main St', 'Jewel', 'Joe Smith'),
(0002, '1+630-255-4788', '711 Smile St', 'Kroger', 'Bill Gates'),
(0003, '1+224-124-7489', '411 Sad St', 'Aldi', 'Jill Bob'),
(0004, '1+847-514-6347', '789 Smith Rd', 'Win-Dixie', 'Barb Santos'),
(0005, '1+630-417-3697', '100 Acre Wood', 'Walmart', 'Piglet Eeyore'),
(0006, '1+847-898-4563', '1725 Slough Avenue', 'Target', 'Michael Scott');


INSERT INTO Product VALUES
(1001, 'Beverage', 'Coca-Cola Can', 1.25, 'per can'),
(1002, 'Beverage', 'Orange Juice', 3.50, 'per bottle'),
(1003, 'Snack', 'Potato Chips', 2.00, 'per pack'),
(1004, 'Grocery', 'Whole Wheat Bread', 2.75, 'per loaf'),
(1005, 'Grocery', 'Brown Rice', 5.00, 'per kg'),
(1006, 'Personal Care', 'Toothpaste', 4.25, 'per tube'),
(1007, 'Electronics', 'AA Batteries', 7.00, 'per pack'),
(1008, 'Stationery', 'Ballpoint Pen', 1.10, 'per piece');

INSERT INTO Includes VALUES
(0001, 1001, 2),
(0001, 1003, 1),
(0002, 1002, 1),
(0003, 1004, 2),
(0004, 1005, 1),
(0005, 1006, 1),
(0006, 1007, 2),
(0007, 1008, 3),
(0008, 1001, 1),
(0008, 1003, 1);

INSERT INTO Restock VALUES
(9001, 0001, 1001, 50, '2025-01-01'),
(9002, 0001, 1003, 30, '2025-01-02'),
(9003, 0002, 1002, 40, '2025-01-03'),
(9004, 0003, 1004, 25, '2025-01-04'),
(9005, 0004, 1005, 60, '2025-01-05'),
(9006, 0005, 1006, 20, '2025-01-05'),
(9007, 0005, 1007, 15, '2025-01-06'),
(9008, 0006, 1008, 80, '2025-01-07'),
(9009, 0004, 1001, 40, '2025-01-08'),
(9010, 0002, 1004, 25, '2025-01-09');
```

- Kellie Clinton
- Sum and Count of Transactions for each Employee per Store
- Calculate the biggest transaction each employee per store will ring up that is more than $5 on 01/01/2025. Count how many transactions each employee processed.

SELECT S.Store_Name, E.Employee_Name, **SUM** (T.Transaction_Total_Amount) **AS** Total_Transaction_Amount, **Count** (E.Employee_ID) **AS** Employee_Wrung_Up_Count

FROM Transaction T, Store S, Employee E

WHERE T.Employee_ID = E.Employee_ID **AND**

T.Store_ID = S.Store_ID **AND**

T.Transaction_Date = '01/01/2025'

GROUP BY S.Store_ID, E.Employee_ID

HAVING SUM(T.Transaction_Total_Amount) > 5

ORDER BY Sum(T.Transaction_Total_Amount) **DESC**;

Query Query History

```
1  SELECT S.Store_Name, E.Employee_Name, SUM (T.Transaction_Total_Amount) AS Tota
2  FROM Transaction T, Store S, Employee E
3  WHERE T.Employee_ID = E.Employee_ID AND
4  T.Store_ID = S.Store_ID AND
5  T.Transaction_Date = '01/01/2025'
6  GROUP BY S.Store_ID, E.Employee_ID
7  HAVING SUM(T.Transaction_Total_Amount) > 5
8  ORDER BY Sum(T.Transaction_Total_Amount) DESC;
9
```

Data Output Messages Notifications

Showing rows: 1 to 5

	store_name character varying (20)	employee_name character varying (20)	total_transaction_amount numeric	employee_wrung_up_count bigint
1	Target	Dan Arnold	131	4
2	Win-Dixie	Nancy Lemon	115	2
3	Win-Dixie	Rachel Lime	112	1
4	Jewel	Julie Pumpkin	60	1
5	Win-Dixie	Tom Schwarts	45	1

- Lingjie Liao
- Transaction Summary with Total Value
- Calculates, for each transaction, the number of distinct products, total quantity, and total value based on product prices.
  Joins *Includes*, *Transaction*, and *Product* tables, groups results by transaction, and displays the top 10 transactions ordered by total value.

Query | Query History

```sql
SELECT
    i.Transaction_ID,
    t.Transaction_Date,
    COUNT(DISTINCT i.Product_ID) AS Num_Products,
    SUM(i.Product_Quantity) AS Total_Items,
    SUM(p.Product_Price * i.Product_Quantity) AS Total_Value,
    STRING_AGG(
        CONCAT(p.Product_Name, ' ($', p.Product_Price, ')'),
        ', '
        ORDER BY p.Product_ID
    ) AS Product_List
FROM Includes i
JOIN Transaction t ON i.Transaction_ID = t.Transaction_ID
JOIN Product p ON i.Product_ID = p.Product_ID
GROUP BY i.Transaction_ID, t.Transaction_Date
ORDER BY Total_Value DESC
LIMIT 8;
```

Data Output | Messages | Notifications

Showing rows: 1 to 8 | Page No: 1 of 1

	transaction_id integer	transaction_date date	num_products bigint	total_items bigint	total_value numeric	product_list text
1	6	2025-01-01	1	2	14.00	AA Batteries ($7.00)
2	3	2025-01-01	1	2	5.50	Whole Wheat Bread ($2.75)
3	4	2025-01-01	1	1	5.00	Brown Rice ($5.00)
4	1	2025-01-01	2	3	4.50	Coca-Cola Can ($1.25), Potato Chips ($2.00)
5	5	2025-01-01	1	1	4.25	Toothpaste ($4.25)
6	2	2025-01-01	1	1	3.50	Orange Juice ($3.50)
7	7	2025-01-01	1	3	3.30	Ballpoint Pen ($1.10)
8	8	2025-01-01	2	2	3.25	Coca-Cola Can ($1.25), Potato Chips ($2.00)

Total rows: 8 | Query complete 00:00:00.031 | LF

- Zubair Zaki
- Product Restock Summary with Total, Value and Average Batch Quantity.

- Find the total restocked quantity and total restocked value (price × quantity), Average Quantity Restocked and total number of restock events for each product type which is sorted by total value Descending order and name is Ascending order

SELECT P.Product_ID, P.Product_Name, P.Product_Price, **SUM** (R.Restock_Quantity) **AS** Total_Quantity, **SUM** (P.Product_Price * R.Restock_Quantity) **AS** Total_Value, **AVG**(r.restock_quantity) **AS** avg_batch_qty, **COUNT** (r.store_id) **AS** restock_events

FROM Restock R

JOIN Product P **ON** P.Product_ID = R.Product_ID

GROUP BY P.Product_ID, P.Product_Type, P.Product_Price

ORDER BY Total_Value **DESC**, P.Product_Name **ASC**;

```
191  SELECT P.Product_ID, P.Product_Name, P.Product_price,
192  SUM (R.Restock_Quantity) AS Total_Quantity,
193  SUM(P.Product_Price * R.Restock_Quantity) AS Total_Value,
194  AVG(r.restock_quantity) AS avg_batch_qty, COUNT(r.store_ID)AS restock_events
195  FROM Product P , Restock R
196  WHERE P.Product_ID = R.Product_ID
197  GROUP BY P.Product_ID, P.Product_Type, P.Product_Name
198  ORDER BY Total_Value DESC, P.Product_Name ASC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 8 | Page No: 1 of 1

	product_id [PK] integer	product_name character varying (40)	product_price numeric	total_quantity bigint	total_value numeric	avg_batch_qty numeric	restock_events bigint
1	1005	Brown Rice	5.00	60	300.00	60.0000000000000000	1
2	1002	Orange Juice	3.50	40	140.00	40.0000000000000000	1
3	1004	Whole Wheat Bread	2.75	50	137.50	25.0000000000000000	2
4	1001	Coca-Cola Can	1.25	90	112.50	45.0000000000000000	2
5	1007	AA Batteries	7.00	15	105.00	15.0000000000000000	1
6	1008	Ballpoint Pen	1.10	80	88.00	80.0000000000000000	1
7	1006	Toothpaste	4.25	20	85.00	20.0000000000000000	1
8	1003	Potato Chips	2.00	30	60.00	30.0000000000000000	1

Total rows: 8 | Query complete 00:00:00.311 | CRLF | Ln

- Abinaya Ramesh
- Customer Spending Summary

- Print the total amount spent and average spending per transaction, the total number of distinct products bought, and the total number of transactions for each customer, which is sorted by total spent in descending order.

Query | Query History | Scratc

```
SELECT c.Customer_Name,c.Customer_Membership,
    ROUND(SUM(p.Product_Price * i.Product_Quantity), 2) AS Total_Spent,
    ROUND(AVG(p.Product_Price * i.Product_Quantity), 2) AS Avg_Spent_Per_Transaction,
    COUNT(DISTINCT i.Product_ID) AS Distinct_Products_Bought,
    COUNT(DISTINCT t.Transaction_ID) AS Total_Transactions
FROM Customer c,Transaction t,Includes i,Product p
WHERE c.Customer_ID = t.Customer_ID AND t.Transaction_ID = i.Transaction_ID
    AND i.Product_ID = p.Product_ID
GROUP BY c.Customer_Name,c.Customer_Membership
ORDER BY Total_Spent DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5 | Page No

	customer_name character varying (50)	customer_membership character varying (20)	total_spent numeric	avg_spent_per_transaction numeric	distinct_products_bought bigint	total_transactions bigint
1	Emma	Silver	14.00	14.00	1	1
2	Alice	Gold	9.50	3.17	3	2
3	Carol	Gold	8.75	2.92	3	2
4	Bob	Silver	6.80	3.40	2	2
5	David	Bronze	4.25	4.25	1	1

7. ORM Implementation

Kellie Clinton

- Create classes for N-N relationship of Hires, Store and Employee

Create Table:

```python
class Base(DeclarativeBase):
    pass

association_SE_Hires = Table(
    "Hires",
    Base.metadata,
    Column("Store_ID", ForeignKey("Store.id"), primary_key=True),
    Column("Employee_ID", ForeignKey("Employee.id"), primary_key=True),
)

class Store(Base):
    __tablename__ = "Store"

    id: Mapped[int] = mapped_column(primary_key=True)
    SName: Mapped[str] = mapped_column(String(20))
    Phone_Number: Mapped[str] = mapped_column(String(20))
    SAddress: Mapped[str] = mapped_column(String(255))
    Owner_Name: Mapped[str] = mapped_column(String(50))
    Employees: Mapped[List[Employee]] = relationship(
        secondary=association_SE_Hires, back_populates="Stores"
    )

    def __repr__(self) -> str: #represents the object as a string
        employeeList = ''
        for e in self.Employees:
            employeeList = employeeList + ' ' + e.EName
        return f"Store(id={self.id!r}, name={self.SName!r}, Employees={employeeList.strip()!r})"

class Employee(Base):
    __tablename__ = "Employee"

    id: Mapped[int] = mapped_column(primary_key=True)
    EName: Mapped[str] = mapped_column(String(20))
    SSN: Mapped[str] = mapped_column(String(20))
    EAddress: Mapped[str] = mapped_column(String(255))
    Stores: Mapped[List[Store]] = relationship(
        secondary=association_SE_Hires, back_populates="Employees"
    )

    def __repr__(self) -> str: #represents the object as a string
        return f"Employee(id={self.id!r}, name={self.EName!r})"

#Create Tables
Base.metadata.create_all(engine)
```

Insert Data:

```
#Insert Data
with Session(engine) as session:
    e1 = Employee(
        EName="Julie Pumpkin",
        SSN="999-88-7777",
        EAddress="202 Broadway St"
    )
    e2 = Employee(
        EName="Nancy Lemon",
        SSN="212-63-4512",
        EAddress="144 Colfax Rd"
    )
    e3 = Employee(
        EName="Tom Schwarts",
        SSN="747-00-8539",
        EAddress="555 Luna Rd"
    )
    e4 = Employee(
        EName="Rachel Lime",
        SSN="963-52-7219",
        EAddress="998 Plum Grove St"
    )
    e5 = Employee(
        EName="Dan Arnold",
        SSN="417-77-8860",
        EAddress="231 Heather Ct"
    )
    e6 = Employee(
        EName="Sally Seashell",
        SSN="551-27-9437",
        EAddress="741 Kennedy Rd"
    )
    e7 = Employee(
        EName="Joe Davis",
        SSN="321-74-8514",
        EAddress="600 Willow Rd"
    )
    e8 = Employee(
        EName="Victoria Beckham",
        SSN="555-67-9147",
        EAddress="1 Hollywood Blvd"
    )
    e9 = Employee(
        EName="Tom Cruise",
        SSN="111-22-3456",
        EAddress="200 Hollywood Blvd"
    )

    s1 = Store(
        SName="Jewel",
        Phone_Number="1+555-123-3456",
        SAddress="123 Main St",
        Owner_Name="Joe Smith",
        Employees=[e1]
    )

    s2 = Store(
        SName="Kroger",
        Phone_Number="1+630-255-4788",
        SAddress="711 Smile St",
        Owner_Name="Bill Gates"
    )
    s3 = Store(
        SName="Target",
        Phone_Number="1+224-124-7489",
        SAddress="411 Sad St",
        Owner_Name="Jill Bob"
    )
    s4 = Store(
        SName="Aldi",
        Phone_Number="1+847-514-6347",
        SAddress="789 Smith Rd",
        Owner_Name="Barb Santos",
        Employees=[e2,e3,e4, e7, e8, e9]
    )
    s5 = Store(
        SName="Win-Dixie",
        Phone_Number="1+630-417-3697",
        SAddress="100 Acre Wood",
        Owner_Name="Piglet Eeyore"
    )
    s6 = Store(
        SName="Walmart",
        Phone_Number="1+847-898-4563",
        SAddress="1725 Slough Avenue",
        Owner_Name="Michael Scott",
    )
    s6 = Store(
        SName="Walmart",
        Phone_Number="1+847-898-4563",
        SAddress="1725 Slough Avenue",
        Owner_Name="Michael Scott",
        Employees=[e5,e6]
    )

    session.add_all([s1, s2, s3, s4, s5, s6])
    session.commit()

session = Session(engine)
stores = session.query(Store)
for s in stores:
    print(s)
```

Query:

```
# Query 1: all Employees of a single store (s4 = "Aldi")
print("## List of Employees##")
#Join Query

result = session.execute(text('SELECT * FROM "Hires"'))
for row in result:
    print(row)

stmt = (
    select(Employee.EName, text("EName"))
    .join(Store)
    .where(Store.SName == "Aldi")
)
for record in session.execute(stmt):
   print(record)
```

Query Output:

```
## List of Employees##
Tom Schwarts
Joe Davis
Rachel Lime
Victoria Beckham
Nancy Lemon
Tom Cruise
```

Lingjie Liao

Create 1-N Tables: Customer and Transaction

```python
class Customer(Base):
    __tablename__  = "customer"
    __table_args__ = {"schema": "supermarket"}

    customer_id   = Column(Integer, primary_key=True)
    customer_name = Column(String)
    customer_age  = Column(Integer)

    transactions  = relationship(
        "Transaction",
        back_populates="customer"
    )

    def __repr__(self):
        return f"Customer(id={self.customer_id}, name='{self.customer_name}')"


class Transaction(Base):
    __tablename__  = "transactions"
    __table_args__ = {"schema": "supermarket"}

    transaction_id           = Column(Integer, primary_key=True)
    transaction_date         = Column(Date)
    transaction_total_amount = Column(Numeric(10, 2))

    customer_id = Column(
        Integer,
        ForeignKey("supermarket.customer.customer_id")
    )

    customer = relationship(
        "Customer",
        back_populates="transactions"
    )

    def __repr__(self):
        return (
            f"Transaction(id={self.transaction_id}, "
            f"amount={self.transaction_total_amount})"
        )

Base.metadata.create_all(engine)
```

Insert Data

```
    t1 = Transaction(
        transaction_id=101,
        transaction_date=date(2025, 1, 10),
        transaction_total_amount=50.75,
        customer=c1
    )

    t2 = Transaction(
        transaction_id=102,
        transaction_date=date(2025, 1, 11),
        transaction_total_amount=23.40,
        customer=c1
    )

    t3 = Transaction(
        transaction_id=103,
        transaction_date=date(2025, 1, 12),
        transaction_total_amount=89.99,
        customer=c2
    )

    t4 = Transaction(
        transaction_id=104,
        transaction_date=date(2025, 1, 13),
        transaction_total_amount=15.20,
        customer=c1
    )

    t5 = Transaction(
        transaction_id=105,
        transaction_date=date(2025, 1, 14),
        transaction_total_amount=44.10,
        customer=c1
    )

    t6 = Transaction(
        transaction_id=106,
        transaction_date=date(2025, 1, 15),
        transaction_total_amount=120.00,
        customer=c2
    )
```

```
    t7 = Transaction(
        transaction_id=107,
        transaction_date=date(2025, 1, 16),
        transaction_total_amount=9.99,
        customer=c2
    )

    t8 = Transaction(
        transaction_id=108,
        transaction_date=date(2025, 1, 17),
        transaction_total_amount=77.77,
        customer=c1
    )

    session.add_all([c1, c2, t1, t2, t3, t4, t5, t6, t7, t8])
    session.commit()
```

Query: List all transactions for each customer

```
print(" List of Transactions ")

with Session(engine) as session:

    stmt = (
        select(
            Customer.customer_name,
            Transaction.transaction_id,
            Transaction.transaction_date,
            Transaction.transaction_total_amount
        )
        .join(Transaction, Customer.customer_id == Transaction.customer_id)
        .order_by(Customer.customer_id, Transaction.transaction_id)
    )

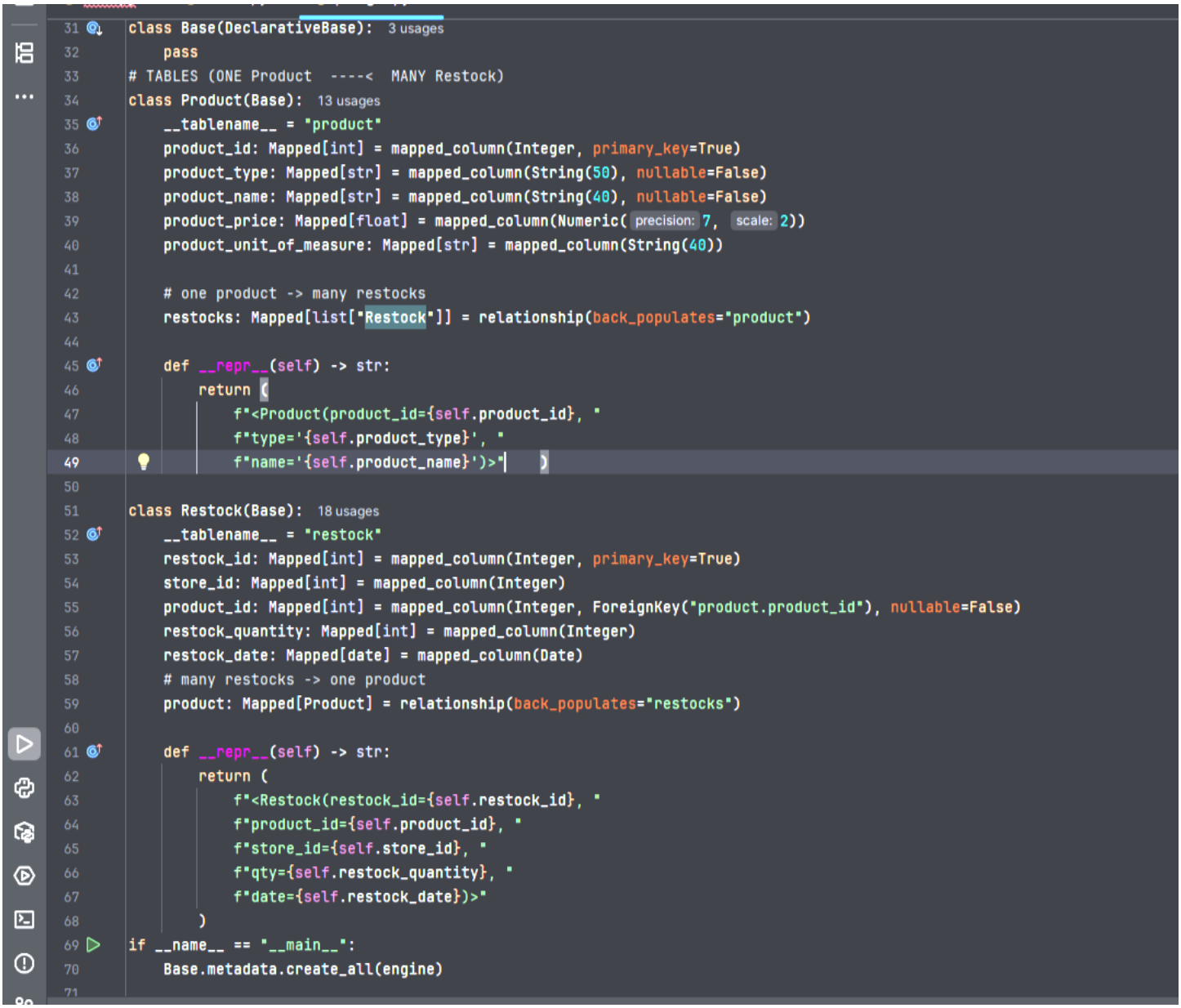
    for record in session.execute(stmt):
        print(record)
```

Output

```
 List of Transactions
('Alice', 101, datetime.date(2025, 1, 10), Decimal('50.75'))
('Alice', 102, datetime.date(2025, 1, 11), Decimal('23.40'))
('Alice', 104, datetime.date(2025, 1, 13), Decimal('15.20'))
('Alice', 105, datetime.date(2025, 1, 14), Decimal('44.10'))
('Alice', 108, datetime.date(2025, 1, 17), Decimal('77.77'))
('Bob', 103, datetime.date(2025, 1, 12), Decimal('89.99'))
('Bob', 106, datetime.date(2025, 1, 15), Decimal('120.00'))
('Bob', 107, datetime.date(2025, 1, 16), Decimal('9.99'))
(ml) liaolingjie@liaolingjiedeMacBook-Pro ~ %
```

Zubair Zaki

Create 1-N tables: Product to restock



```python
class Base(DeclarativeBase):  3 usages
    pass
# TABLES (ONE Product  ----<  MANY Restock)
class Product(Base):  13 usages
    __tablename__ = "product"
    product_id: Mapped[int] = mapped_column(Integer, primary_key=True)
    product_type: Mapped[str] = mapped_column(String(50), nullable=False)
    product_name: Mapped[str] = mapped_column(String(40), nullable=False)
    product_price: Mapped[float] = mapped_column(Numeric( precision: 7,  scale: 2))
    product_unit_of_measure: Mapped[str] = mapped_column(String(40))

    # one product -> many restocks
    restocks: Mapped[list["Restock"]] = relationship(back_populates="product")

    def __repr__(self) -> str:
        return (
            f"<Product(product_id={self.product_id}, "
            f"type='{self.product_type}', "
            f"name='{self.product_name}')>"    )

class Restock(Base):  18 usages
    __tablename__ = "restock"
    restock_id: Mapped[int] = mapped_column(Integer, primary_key=True)
    store_id: Mapped[int] = mapped_column(Integer)
    product_id: Mapped[int] = mapped_column(Integer, ForeignKey("product.product_id"), nullable=False)
    restock_quantity: Mapped[int] = mapped_column(Integer)
    restock_date: Mapped[date] = mapped_column(Date)
    # many restocks -> one product
    product: Mapped[Product] = relationship(back_populates="restocks")

    def __repr__(self) -> str:
        return (
            f"<Restock(restock_id={self.restock_id}, "
            f"product_id={self.product_id}, "
            f"store_id={self.store_id}, "
            f"qty={self.restock_quantity}, "
            f"date={self.restock_date})>"
        )
if __name__ == "__main__":
    Base.metadata.create_all(engine)
```

Insert Data

```python
    # -------- INSERT DATA
    with Session(engine) as session:
        # ----- Products -----
        p1 = Product(
            product_id=1001,
            product_type="Beverage",
            product_name="Coca-Cola Can",
            product_price=1.25,
            product_unit_of_measure="per can",
        )
        p2 = Product(
            product_id=1002,
            product_type="Beverage",
            product_name="Orange Juice",
            product_price=3.50,
            product_unit_of_measure="per bottle",
        )
        p3 = Product(
            product_id=1003,
            product_type="Snack",
            product_name="Potato Chips",
            product_price=2.00,
            product_unit_of_measure="per pack",
        )
        p4 = Product(
            product_id=1004,
            product_type="Grocery",
            product_name="Whole Wheat Bread",
            product_price=2.75,
            product_unit_of_measure="per loaf",
        )
        p5 = Product(
            product_id=1005,
            product_type="Grocery",
            product_name="Brown Rice",
            product_price=5.00,
            product_unit_of_measure="per kg",
        )
        p6 = Product(
            product_id=1006,
            product_type="Personal Care",
            product_name="Toothpaste",
            product_price=4.25,
            product_unit_of_measure="per tube",
        )
        p7 = Product(
            product_id=1007,
            product_type="Electronics",
            product_name="AA Batteries",
            product_price=7.00,
            product_unit_of_measure="per pack",
        )
        p8 = Product(
            product_id=1008,
            product_type="Stationery",
            product_name="Ballpoint Pen",
            product_price=1.10,
            product_unit_of_measure="per piece",
        )

        session.add_all([p1, p2, p3, p4, p5, p6, p7, p8])
        session.commit()

        # ----- Restocks -----
        r1 = Restock(
            restock_id=9001,
            store_id=1,
            product_id=1001,
            restock_quantity=50,
            restock_date=date( year: 2025,  month: 1,  day: 1),
        )
        r2 = Restock(
            restock_id=9002,
            store_id=1,
            product_id=1003,
            restock_quantity=30,
            restock_date=date( year: 2025,  month: 1,  day: 2),
```

```
        r3 = Restock(
            restock_id=9003,
            store_id=2,
            product_id=1002,
            restock_quantity=40,
            restock_date=date( year: 2025,  month: 1,  day: 3),
        )
        r4 = Restock(
            restock_id=9004,
            store_id=3,
            product_id=1004,
            restock_quantity=25,
            restock_date=date( year: 2025,  month: 1,  day: 4),
        )
        r5 = Restock(
            restock_id=9005,
            store_id=4,
            product_id=1005,
            restock_quantity=60,
            restock_date=date( year: 2025,  month: 1,  day: 5),
        )
        r6 = Restock(
            restock_id=9006,
            store_id=5,
            product_id=1006,
            restock_quantity=20,
            restock_date=date( year: 2025,  month: 1,  day: 5),
        )
        r7 = Restock(
            restock_id=9007,
            store_id=5,
            product_id=1007,
            restock_quantity=15,
            restock_date=date( year: 2025,  month: 1,  day: 6),
        )
```

```
        r8 = Restock(
            restock_id=9008,
            store_id=6,
            product_id=1008,
            restock_quantity=80,
            restock_date=date( year: 2025,  month: 1,  day: 7),
        )
        r9 = Restock(
            restock_id=9009,
            store_id=4,
            product_id=1001,
            restock_quantity=40,
            restock_date=date( year: 2025,  month: 1,  day: 8),
        )
        r10 = Restock(
            restock_id=9010,
            store_id=2,
            product_id=1004,
            restock_quantity=25,
            restock_date=date( year: 2025,  month: 1,  day: 9),
        )

        session.add_all([r1, r2, r3, r4, r5, r6, r7, r8, r9, r10])
        session.commit()
```

Query: List of All the Restocks with product details

```python
# -------- QUERY: list all restocks with product details --------
with Session(engine) as session:
    stmt = (
        select(
            Restock.restock_id,
            Restock.restock_date,
            Restock.restock_quantity,
            Restock.store_id,
            Product.product_name,
            Product.product_type,
        )
        .join(Product, Restock.product_id == Product.product_id)
        .order_by(Restock.restock_date, *clauses: Restock.restock_id)
    )

    rows = session.execute(stmt).all()

    print("Restock Activity Report:")
    for restock_id, rdate, qty, store_id, pname, ptype in rows:
        print(
            f"Restock ID: {restock_id}, "
            f"Date: {rdate}, "
            f"Store: {store_id}, "
            f"Product: {pname} ({ptype}), "
            f"Quantity: {qty}"
        )
```

Output

```
C:\Users\zakiz\PycharmProjects\SQLAlchemyassighment3\.venv\Scripts\python.exe C:\Users\zakiz\PycharmProjects\SQLAlchemyassighment3\postgre.py
Restock Activity Report:
Restock ID: 9001, Date: 2025-01-01, Store: 1, Product: Coca-Cola Can (Beverage), Quantity: 50
Restock ID: 9002, Date: 2025-01-02, Store: 1, Product: Potato Chips (Snack), Quantity: 30
Restock ID: 9003, Date: 2025-01-03, Store: 2, Product: Orange Juice (Beverage), Quantity: 40
Restock ID: 9004, Date: 2025-01-04, Store: 3, Product: Whole Wheat Bread (Grocery), Quantity: 25
Restock ID: 9005, Date: 2025-01-05, Store: 4, Product: Brown Rice (Grocery), Quantity: 60
Restock ID: 9006, Date: 2025-01-05, Store: 5, Product: Toothpaste (Personal Care), Quantity: 20
Restock ID: 9007, Date: 2025-01-06, Store: 5, Product: AA Batteries (Electronics), Quantity: 15
Restock ID: 9008, Date: 2025-01-07, Store: 6, Product: Ballpoint Pen (Stationery), Quantity: 80
Restock ID: 9009, Date: 2025-01-08, Store: 4, Product: Coca-Cola Can (Beverage), Quantity: 40
Restock ID: 9010, Date: 2025-01-09, Store: 2, Product: Whole Wheat Bread (Grocery), Quantity: 25

Process finished with exit code 0
```

ABINAYA RAMESH

Create 1-N tables: Transaction and Store.

```python
class Store(Base):
    __tablename__ = "store"

    store_id: Mapped[int] = mapped_column(Integer, primary_key=True)
    store_phone_number: Mapped[str] = mapped_column(String(50))
    store_address: Mapped[str] = mapped_column(String(200))
    store_name: Mapped[str] = mapped_column(String(100))
    store_owner_name: Mapped[str] = mapped_column(String(100))

    transactions: Mapped[List["Transaction"]] = relationship(
        back_populates="store", cascade="all, delete-orphan"
    )

    def __repr__(self) -> str:
        return f"Store(ID={self.store_id}, Name={self.store_name}, Owner={self.store_owner_name})"

class Transaction(Base):
    __tablename__ = "transaction"

    transaction_id: Mapped[int] = mapped_column(Integer, primary_key=True)
    transaction_date: Mapped[Optional[Date]] = mapped_column(Date)
    transaction_workstation: Mapped[str] = mapped_column(String(20))
    transaction_total_amount: Mapped[float] = mapped_column(Float)
    transaction_payment_method: Mapped[str] = mapped_column(String(30))

    store_id: Mapped[int] = mapped_column(ForeignKey("store.store_id"))

    store: Mapped["Store"] = relationship(back_populates="transactions")

    def __repr__(self) -> str:
        return f"Transaction(ID={self.transaction_id}, Amount={self.transaction_total_amount})"

# Create Tables
Base.metadata.create_all(engine)
```

Insert Data

```python
with Session(engine) as session:

    # Store Data
    s1 = Store(store_id=1, store_phone_number="1+555-123-3456",
               store_address="123 Main St", store_name="Jewel",
               store_owner_name="Joe Smith")
    s2 = Store(store_id=2, store_phone_number="1+630-255-4788",
               store_address="711 Smile St", store_name="Kroger",
               store_owner_name="Bill Gates")
    s3 = Store(store_id=3, store_phone_number="1+224-124-7489",
               store_address="411 Sad St", store_name="Aldi",
               store_owner_name="Jill Bob")
    s4 = Store(store_id=4, store_phone_number="1+847-514-6347",
               store_address="789 Smith Rd", store_name="Win-Dixie",
               store_owner_name="Barb Santos")
    s5 = Store(store_id=5, store_phone_number="1+630-417-3697",
               store_address="100 Acre Wood", store_name="Walmart",
               store_owner_name="Piglet Eeyore")
    s6 = Store(store_id=6, store_phone_number="1+847-898-4563",
               store_address="1725 Slough Avenue", store_name="Target",
               store_owner_name="Michael Scott")

    session.add_all([s1, s2, s3, s4, s5, s6])

    # Transaction Data
    t1 = Transaction(transaction_id=1, transaction_date=date(2025,1,1),
                     transaction_workstation="WS-001",
                     transaction_total_amount=60,
                     transaction_payment_method="Credit Card", store_id=1)
    t2 = Transaction(transaction_id=2, transaction_date=date(2025,1,1),
                     transaction_workstation="WS-002",
                     transaction_total_amount=10,
                     transaction_payment_method="Check",store_id=2)
    t3 = Transaction(transaction_id=3, transaction_date=date(2025,1,1),
                     transaction_workstation="WS-003",
                     transaction_total_amount=45,
                     transaction_payment_method="Credit Card", store_id=3)
```

```
t4 = Transaction(transaction_id=4, transaction_date=date(2025,1,1),
                 transaction_workstation="WS-004",
                 transaction_total_amount=105,
                 transaction_payment_method="Credit Card", store_id=1)
t5 = Transaction(transaction_id=5, transaction_date=date(2025,1,1),
                transaction_workstation="WS-005",
                transaction_total_amount=25,
                transaction_payment_method="Direct Deposit", store_id=4)
t6 = Transaction(transaction_id=6, transaction_date=date(2025,1,1),
                 transaction_workstation="WS-005",
                 transaction_total_amount=3,
                 transaction_payment_method="Credit Card", store_id=5)
t7 = Transaction(transaction_id=7, transaction_date=date(2025,1,1),
                 transaction_workstation="WS-003",
                 transaction_total_amount=112,
                 transaction_payment_method="Check",store_id=2)
t8 = Transaction(transaction_id=8, transaction_date=date(2025,1,1),
                 transaction_workstation="WS-005",
                 transaction_total_amount=56,
                 transaction_payment_method="Cash",store_id=3)
t9 = Transaction(transaction_id=9, transaction_date=date(2025,1,1),
                 transaction_workstation="WS-001",
                 transaction_total_amount=47,
                 transaction_payment_method="Credit Card", store_id=5)
t10 = Transaction(transaction_id=10,transaction_date=date(2025,1,2),
                  transaction_workstation="WS-002",
                  transaction_total_amount=87,
                  transaction_payment_method="Cash",store_id=6)

session.add_all([t1, t2, t3, t4, t5, t6, t7, t8, t9, t10])
session.commit()
```

Query: List all transactions paid by Credit Card.

```python
# QUERIES 1:

session = Session(engine)

stmt = (
    select(Transaction)
    .join(Transaction.store)
    .where(Transaction.transaction_payment_method == "Credit Card")
)

results = session.scalars(stmt).all()

print("Transactions paid by Credit Card:")
for t in results:
    print(
        f"Transaction ID: {t.transaction_id}, "
        f"Amount: {t.transaction_total_amount}, "
        f"Store: {t.store.store_name}"
    )
```

Output:

```
Transactions paid by Credit Card:
Transaction ID: 4, Amount: 105.0, Store: Jewel
Transaction ID: 1, Amount: 60.0, Store: Jewel
Transaction ID: 4, Amount: 105.0, Store: Jewel
Transaction ID: 1, Amount: 60.0, Store: Jewel
Transaction ID: 1, Amount: 60.0, Store: Jewel
Transaction ID: 3, Amount: 45.0, Store: Aldi
Transaction ID: 3, Amount: 45.0, Store: Aldi
Transaction ID: 9, Amount: 47.0, Store: Walmart
Transaction ID: 9, Amount: 47.0, Store: Walmart
Transaction ID: 6, Amount: 3.0, Store: Walmart
```

GROUP STATUS REPORT

GROUP #: 1 **GROUP NAME**: Dashers **PHASE** #: 3

Dates & attendance at group meetings in this phase:

Thursday,	Nov 06	12:00 - 12:30 pm	All group members present
Wednesday, Oct 22		1:00 - 2:00 pm	Lingjie Liao & Kellie Clinton & Prof

Overview of progress on project as of <November 06>

Discussed who will do which entity relationship. The team will meet with the professor on Tuesday, 11/11 to review the code. After class that day, the team will meet to discuss the presentation.

CONTRIBUTIONS OF GROUP MEMBERS

Leader: Abinaya Ramesh

- Organized and compiled the team's meeting reports and progress notes.
- Create classes for 1-N relationship of Customer and Transaction

Recorder: Zubair Zaki

- Check all sections of the report to ensure it was ready for submission.
- Create classes for 1-N relationship of Product and Restock

Phase Checker: Lingjie Liao

- Format Queries section and included it in the final documentation.
- Create classes for 1-N relationship of Customer and Transaction

Technical Advisor: Kellie Clinton

- Coordinated the overall formatting and consistency of the final report.
- Create classes for N-N relationship of Hires, Store and Employee